Bases de datos relacionales · Práctica de bases de datos

Teoría: resumen

1. **Sistemas de información**: Sistema de información · Componentes de un sistema de información · Pirámide de un sistema de información
2. **Ficheros frente a bases de datos**: Problemas de los sistemas basados en ficheros · Qué aporta una base de datos · Esquema, sub-esquemas y lenguajes (DDL / DML)
3. **El SGBD**: El SGBD y su arquitectura · Propiedades de un SGBD · Actores y roles
4. **Transacciones (ACID)**: Transacciones y atomicidad · Consistencia · Aislamiento · Durabilidad
5. **Niveles de abstracción**: La arquitectura ANSI/X3/SPARC · Independencia lógica y física
6. **Almacenamiento y eficiencia**: Ficheros, registros y registros de longitud fija · Registros de longitud variable · Soportes de almacenamiento y bloques · Tiempo de respuesta: Ts = α + βb
7. **Organización de ficheros e índices**: Las organizaciones de ficheros de un vistazo · Organización secuencial (Sequential File Organization) · Organización heap (Heap File Organization) · Organización hash (Hash File Organization) · Organización en árbol B+ (B+ tree File Organization) · Organización agrupada (Clustered File Organization) · ISAM (Indexed Sequential Access Method) · Cómo elegir la organización de ficheros
8. **Ciclo de vida de los datos**: Modelos de datos y para qué sirven · El ciclo de vida del modelado de datos

Sistemas de información

Sistema de información

Conjunto de elementos ordenadamente relacionados entre sí, que cumplen reglas y aportan a la organización la información necesaria para el cumplimiento de sus fines.

- **Completitud**: tener toda la información necesaria, que no falte nada: información de datos y funcional (por ejemplo, reglas de negocio).
- **Fiabilidad**: el sistema funciona correctamente y de manera consistente. Tiene tres facetas: **exactitud de datos** (la información almacenada es correcta y los cálculos no son erróneos), **disponibilidad** (el sistema está operativo el mayor tiempo posible, pactado en un SLA, Service Level Agreement) y **consistencia** (los mismos datos aparecen igual en distintas partes del sistema y las reglas de negocio se aplican de manera uniforme).

> **Ejemplo.** Un SLA de disponibilidad del **99,999 %** deja un 0,001 % del año para fallos: 0,00001 × 525.600 minutos ≈ **5,26 minutos de caída al año**. Un SLA del 99,9 % permitiría unas 8,76 horas.

> **Error frecuente.** Tratar datos e información como sinónimos. "Saldo = 1.250" es un dato; "este cliente lleva seis meses con saldo inferior a 2.000, así que ofrécele una tarjeta de crédito" es información.

Componentes de un sistema de información

Un sistema de información se compone de contenidos, equipo físico, equipo lógico y equipo humano.

- **Contenidos**: datos e información internos y/o externos a la organización.
- **Equipo físico**: servidores, redes, almacenamiento. **On premise** (local, compra de hardware) o **cloud** (pay as you go).
- **Equipo lógico**: arquitecturas, cloud computing y software especializado, por ejemplo una **arquitectura multicapa** con capas independientes frente a **SOA** (Service Oriented Architecture, microservicios).
- **Equipo humano (recursos humanos)**: usuarios del sistema, DBA (administrador de la base de datos), data engineer, data analyst, data architect y data scientist.

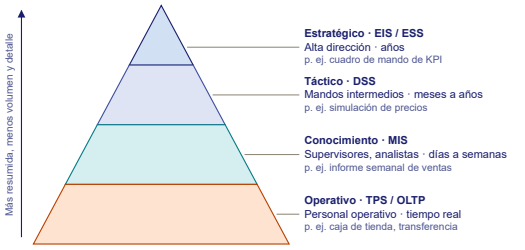
> **Ejemplo.** Una librería online: **contenidos** = catálogo, clientes, pedidos y las tarifas de las editoriales; **equipo físico** = dos servidores de base de datos en la nube alquilados por horas; **equipo lógico** = la tienda web, un microservicio de pagos y el SGBD; **equipo humano** = clientes y empleados (usuarios), un DBA, un data engineer que carga cada noche los ficheros de las editoriales y un analista que construye el cuadro de mando de ventas.

Pirámide de un sistema de información

Niveles jerárquicos de información en una organización: a medida que subimos de nivel, la información se resume cada vez más.

Niveles de la pirámide, con un banco y una cadena de tiendas como ejemplo

Nivel	Volumen · detalle · uso · horizonte	Banco	Cadena de tiendas
Operativo (TPS/OLTP)	Muy alto · máximo · continuo · tiempo real	Registra cada pago con tarjeta y cada retirada en cajero en el instante en que ocurre	Cada venta en caja actualiza el stock de la tienda
Conocimiento (MIS)	Alto · medio · diario/semanal · días/semanas	Informe diario de pagos y descubiertos por oficina	Informe semanal de ventas y roturas de stock por tienda
Táctico (DSS)	Medio · bajo · mensual · meses/años	Simula el efecto de subir 0,5 puntos el tipo de las hipotecas	Analiza qué productos promocionar el próximo trimestre
Estratégico (EIS/ESS)	Bajo · mínimo · trimestral · años	Cuadro de mando del consejo: cuota de mercado y riesgo por país	Decide si abrir 20 tiendas en Portugal



Cada nivel resume el inferior: menos datos, más agregados y con un horizonte temporal más largo a medida que subes.

> **Ejemplo.** Un pago con tarjeta de 42,50 € en un supermercado es una sola fila OLTP. El MIS suma el día (1,2 millones de pagos); el DSS compara meses; el EIS le muestra al consejo un único número: los ingresos por tarjeta han crecido un 6 % este año.

Ficheros frente a bases de datos

Problemas de los sistemas basados en ficheros

Gestionar un sistema de información con ficheros sueltos (xlsx, txt, csv o los que sean) provoca redundancia, inconsistencia, dependencia de la estructura, falta de integridad y acceso limitado.

- **Redundancia**: los datos pueden estar repetidos en diferentes archivos.
- **Inconsistencia**: puedo referirme a una cosa de formas diferentes (en un archivo "Alberto López", en otro "A. López"), o se actualiza una copia y la otra no.
- **Aislamiento de los datos**: datos repartidos en ficheros con formatos distintos son difíciles de combinar en una consulta.
- **Dependencia de la estructura de los archivos**: los programas conocen el formato exacto de cada fichero; añadir un campo obliga a reescribir todos los programas que lo leen.
- **Sin integridad de datos** al no haber validaciones: se acepta una edad de -20 o la venta de un producto que no existe.
- **Acceso limitado (concurrencia)**: solo una aplicación o un usuario puede modificar a la vez.

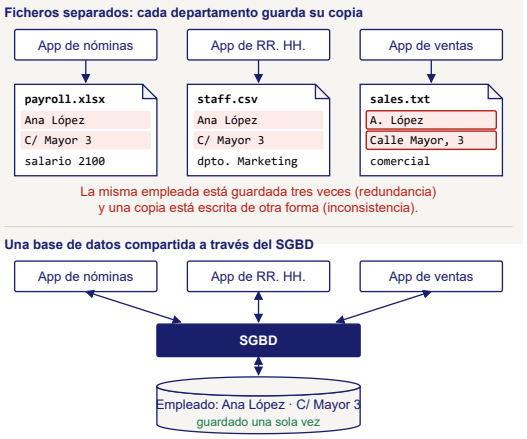
> **Ejemplo.** Recursos Humanos mantiene `empleados_rrhh.csv` y Nóminas mantiene `empleados_nominas.xlsx`, ambos con la dirección del empleado. Ana López se muda; RR. HH. actualiza su fichero y Nóminas no. En marzo su nómina llega a la dirección antigua: la **redundancia** (dos copias) ha provocado una **inconsistencia** (dos verdades distintas). Cruzar ambos ficheros para listar "salario por departamento" exige un script hecho a mano (**aislamiento**), y cuando Nóminas añade una columna IBAN, el programa de informes de RR. HH. falla (**dependencia**).

Qué aporta una base de datos

Una base de datos es un conjunto de datos relacionados que sustentan entre sí el sistema de información de una organización: exhaustivos, no redundantes y estructurados.

- **Sin redundancia**: un dato se guarda una sola vez.
- **Consistencia**: datos uniformes para todos.
- **Independencia**: los cambios de estructura no afectan a los programas.
- **Acceso concurrente**: varios usuarios a la vez.
- **Integridad**: validaciones automáticas y relaciones entre los datos.

Ejemplo. RR. HH. y Nóminas comparten ahora una única tabla `empleados` con una fila para Ana López (id 1043). Cuando RR. HH. cambia su dirección, Nóminas lee la nueva en la siguiente consulta, porque solo hay una copia. Una regla `CHECK (salario > 0)` rechaza un salario negativo venga de la aplicación que venga, y ambos departamentos pueden modificar empleados distintos en el mismo instante.



Arriba: cada departamento guarda su propia copia de los datos. Abajo: todas las aplicaciones pasan por un único SGBD hasta una sola copia compartida.

Esquema, sub-esquemas y lenguajes (DDL / DML)

El esquema es la organización conceptual de la base de datos; los sub-esquemas son las partes que ve cada usuario o aplicación; el DDL los define y el DML trabaja con los datos.

Comandos DDL (estructura)

Comando	Qué hace	Sintaxis
`CREATE`	Crea una tabla y sus columnas junto con su tipo de datos	`CREATE TABLE`
`ALTER`	Modifica los nombres de las columnas y añade o elimina una columna	`ALTER TABLE`
`RENAME`	Cambia el nombre de la tabla	`RENAME TABLE`
`COMMENT`	Añade una explicación al código SQL para que la revisen otros miembros del equipo	`--` o `/* ... */`
`TRUNCATE`	Elimina los datos de una tabla sin borrar la tabla	`TRUNCATE TABLE`
`DROP`	Elimina la tabla con sus datos	`DROP TABLE`

```
-- DDL: definir la estructura
CREATE TABLE empleados (
  id           INT           PRIMARY KEY,
  nombre       VARCHAR(50)   NOT NULL,
  departamento INT,
  salario      DECIMAL(10,2)
);

-- DML: trabajar con los datos
INSERT INTO empleados (id, nombre, departamento, salario)
VALUES (1, 'Ana', 10, 2000.00),
       (2, 'Luis', 10, 2500.00),
       (3, 'Marta', 20, 3000.00);

UPDATE empleados SET salario = salario * 1.05 WHERE departamento = 10;

SELECT nombre, salario FROM empleados WHERE departamento = 10;
-- Ana 2100.00, Luis 2625.00
```

Ejemplo. Un sub-esquema para la aplicación de recepción de un gimnasio podría ser una vista con solo el `nombre` y el `estado_cuota` de cada socio: recepción nunca ve los datos bancarios. El `UPDATE` de arriba sube el sueldo solo al departamento 10; sin `WHERE` se lo subiría a los tres empleados.

Error frecuente. Confundir `TRUNCATE`/`DELETE` con `DROP`. Tras `TRUNCATE TABLE empleados` la tabla vacía sigue existiendo; tras `DROP TABLE empleados` hay que volver a crearla.

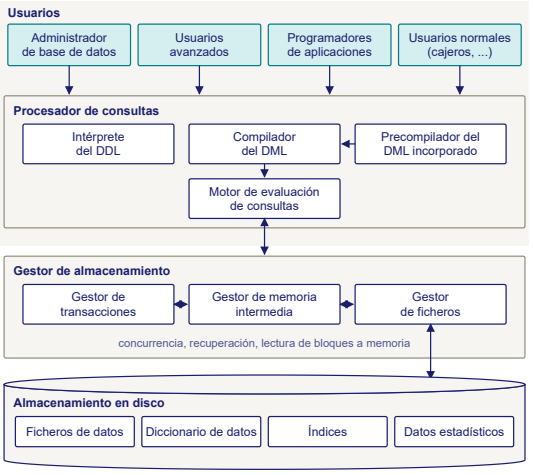
El SGBD

El SGBD y su arquitectura

Un SGBD es un conjunto coordinado de programas, procedimientos, lenguajes, etc. que suministra, tanto a los usuarios no informáticos como a los técnicos especializados en el tratamiento de datos, los medios necesarios para describir, recuperar y manipular los datos almacenados en la base, manteniendo su integridad, confidencialidad y seguridad.

- Gestión de la seguridad; instalación y configuración de la base de datos; gestión de la comunicación de la base de datos.
- Creación y especificación del diccionario de datos; administrar y crear la estructura física de la base de datos.
- Manipulación de datos y creación de aplicaciones de usuario.
- Exportación e importación de datos a/o desde otros sistemas.
- Creación y restablecimiento de copias de seguridad; recuperación en caso de desastre.

> **Ejemplo.** La aplicación de un cajero ejecuta `SELECT saldo FROM cuentas WHERE cuenta_id = 'A123'`. El compilador del DML consulta el diccionario de datos (la tabla y la columna existen), usa los datos estadísticos para elegir el índice sobre `cuenta_id`, el motor de evaluación pide el bloque al gestor de memoria intermedia y el gestor de ficheros solo lo lee del disco si no está ya en memoria.



Usuarios arriba, el SGBD en el centro (procesador de consultas y gestor de almacenamiento) y el disco abajo (diccionario de datos, datos estadísticos, índices y ficheros de datos).

Propiedades de un SGBD

Un SGBD es autodescriptivo, aísla programas y datos, ofrece una visión abstracta y varias vistas de los datos, comparte los datos entre usuarios y aporta seguridad y eficiencia.

- **Autodescriptivo**: para la definición de la BD se utiliza una BD especial denominada **catálogo**, que contiene la estructura de la BD: **metadatos**.
- **Aislamiento entre programas y datos**: la estructura de la BD está separada de los programas; es posible modificar la BD sin necesidad de modificar (todos) los programas que acceden a ella.
- **Visión abstracta de los datos**: una representación conceptual que oculta los detalles de almacenamiento o de implementación de las operaciones.
- **Varias vistas de los datos**: un subconjunto (para un profesor, solo los alumnos matriculados en la asignatura que imparte), una vista parcial (según el tipo de usuario, ocultando datos confidenciales) o datos virtuales derivados de otros (totales, resúmenes, etc. que no están almacenados en la BD).
- **Compartición de datos entre múltiples usuarios**: un único almacén; las **transacciones** (unidades de trabajo que agrupan una o más operaciones y garantizan las propiedades ACID) controlan los accesos concurrentes.
- **Seguridad** frente a accesos no autorizados y fallos del sistema.
- **Eficiencia**: validaciones, accesos simultáneos, etc.

```
-- El catálogo también se puede consultar (metadatos, no datos de usuario)
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'empleados';

-- Una vista: un subconjunto más datos derivados que no se almacenan
CREATE VIEW nomina_departamento AS
SELECT departamento, COUNT(*) AS plantilla, SUM(salario) AS total
```

```
FROM empleados
GROUP BY departamento;
```

> **Ejemplo.** Consultar el catálogo para `empleados` devuelve filas como (`id`, `int`) y (`salario`, `decimal`): datos sobre los datos. La vista `nomina_departamento` devuelve (10, 2, 4725.00) aunque ninguna tabla guarda el número 4725.

Actores y roles

Mucha gente trabaja con una base de datos, cada uno por una puerta distinta: desde los usuarios finales que solo ven una aplicación hasta el DBA que mantiene vivo todo el sistema.

Quién hace qué, con un ejemplo del día a día

Rol	Qué hace	Un día cualquiera
DBA (administrador de la base de datos)	Cambios de esquema, almacenamiento físico, usuarios y permisos, copias de seguridad, recuperación, optimización	En un hospital, programa la copia de seguridad nocturna a las 02:00, concede a la aplicación de recepción solo `SELECT` sobre `Pacientes` y añade un índice porque la búsqueda de citas tarda 4 s
Diseñador de BD / data architect	Diseño conceptual y lógico; el arquitecto decide además la plataforma de datos global	Dibuja el modelo E/R de un nuevo programa de fidelización: CLIENTE, TARJETA, MOVIMIENTO_PUNTOS, y decide que va en el mismo clúster PostgreSQL que las ventas
Desarrollador de aplicaciones	Programa aplicaciones que incorporan DML	Programa el botón "reservar clase" de la app de un gimnasio: `INSERT INTO reservas ...` dentro de una transacción que también resta las plazas libres
Data engineer	Construye procesos ETL que mueven y transforman datos entre sistemas	Carga cada noche los 300.000 pagos con tarjeta del día desde la BD OLTP al data warehouse y arregla el proceso cuando un proveedor cambia el formato de su CSV
Data analyst	Consulta y resume datos para tomar decisiones (un usuario avanzado)	Escribe una consulta con `GROUP BY` que muestra que la venta de paraguas crece un 40 % las semanas de lluvia y la añade al cuadro de mando semanal
Data scientist	Construye modelos estadísticos y de machine learning sobre los datos	Entrena un modelo de abandono con dos años de visitas de los socios para predecir quién se dará de baja del gimnasio
Usuario final	Usa aplicaciones con consultas preconfiguradas; nunca escribe SQL	Un cliente del banco consulta su saldo en la app; una recepcionista busca a un paciente por su DNI

> **Ejemplo.** En el hospital anterior, la recepcionista (usuaria final) puede leer `Pacientes`, pero cuando intenta abrir `Diagnosticos` el SGBD se lo impide, porque el DBA solo concedió al rol de recepción `SELECT` sobre `Pacientes`.

> **Error frecuente.** Pensar que el DBA diseña el modelo conceptual o programa las aplicaciones. El DBA es responsable del esquema en producción, la organización física, los permisos y el mantenimiento; el diseño y el código de las aplicaciones corresponden a otros roles.

Transacciones (ACID)

Transacciones y atomicidad

Una transacción agrupa varias operaciones en una unidad de trabajo. Atomicidad: todo se ejecuta o nada se ejecuta.

- Paso 1: `BEGIN TRANSACTION`. A123 tiene 5.000 €, B456 tiene 2.000 €.
- Paso 2: el primer `UPDATE` descuenta 1.000 € de A123 (4.000 €, todavía sin confirmar).
- Paso 3: el segundo `UPDATE` falla: la cuenta B456 no existe, se va la luz...
- Paso 4: el SGBD revierte automáticamente el primer `UPDATE`. Al reiniciar, A123 vuelve a tener 5.000 € y B456 2.000 €: nunca te quedarás con el dinero descontado de una cuenta sin haberse acreditado en la otra.
- Si el paso 3 tiene éxito, el `COMMIT` valida los dos cambios a la vez: A123 4.000 €, B456 3.000 €.

```
BEGIN TRANSACTION;
UPDATE cuentas SET saldo = saldo - 1000 WHERE cuenta_id = 'A123';
UPDATE cuentas SET saldo = saldo + 1000 WHERE cuenta_id = 'B456';
COMMIT;

-- Otro ejemplo: un pedido, su stock y su factura
BEGIN TRANSACTION;
INSERT INTO pedidos (cliente_id, total) VALUES (123, 500);
```

```
UPDATE inventario SET stock = stock - 5 WHERE producto_id = 'P001';
INSERT INTO facturacion (pedido_id, monto) VALUES (LAST_INSERT_ID(), 500);
COMMIT;
```

> **Ejemplo.** En el ejemplo del pedido, si falla el `INSERT` de la factura, el pedido del cliente 123 desaparece y las 5 unidades vuelven al stock: nunca hay un pedido sin factura.

Consistencia

Los datos mantienen su integridad: una transacción lleva la base de datos de un estado válido a otro, y el SGBD rechaza las operaciones que violan sus reglas.

- Regla: `CHECK (saldo >= 0)` en `cuentas`.
- Paso 1: A123 tiene 600 €. Empieza una transferencia de 1.000 € a B456.
- Paso 2: el cargo dejaría A123 en -400 €: el SGBD lanza un error.
- Paso 3: la transacción se revierte; A123 conserva sus 600 € y B456 no cambia. El dinero total del banco (600 + 2.000) es el mismo antes y después.

```
-- Esta transacción FALLA porque viola la regla de edad positiva
BEGIN TRANSACTION;
UPDATE alumnos SET edad = -20 WHERE alumno_id = 'A123';
-- ERROR: la base de datos rechaza esta operación
ROLLBACK;

-- Integridad referencial: no puedes eliminar un cliente que tiene pedidos activos
DELETE FROM clientes WHERE cliente_id = 123;
-- ERROR: viola la clave foránea en la tabla pedidos
```

> **Ejemplo.** El cliente 123 tiene dos pedidos abiertos. `DELETE FROM clientes WHERE cliente_id = 123` se rechaza, porque si no los pedidos apuntarían a un cliente que ya no existe.

Aislamiento

Las transacciones no se interfieren entre sí: una transacción no completada (con commit) es invisible al resto del mundo.

- A123 tiene 1.000 €. La transferencia T1 (300 € a B456) y la transferencia T2 (500 € a C789) empiezan a la vez.
- Sin aislamiento: T1 lee 1.000, T2 lee 1.000; T1 escribe 700, T2 escribe 500. Saldo final 500 €: el cargo de T1 se ha **perdido** y el banco ha regalado 300 €.
- Con aislamiento: el `UPDATE` de T2 sobre A123 espera (queda **bloqueado**) hasta que T1 hace commit; entonces lee 700 y escribe 200. Saldo final 200 € = 1.000 - 300 - 500.

```
-- Usuario A
BEGIN TRANSACTION;
SELECT stock FROM productos WHERE id = 'P001';   -- ve stock = 1
-- tarda un rato en decidirse...
UPDATE productos SET stock = 0 WHERE id = 'P001';
INSERT INTO pedidos (producto_id, cantidad) VALUES ('P001', 1);
COMMIT;

-- Usuario B, a la vez
BEGIN TRANSACTION;
SELECT stock FROM productos WHERE id = 'P001';   -- también ve stock = 1
UPDATE productos SET stock = 0 WHERE id = 'P001'; -- BLOQUEADO hasta que A termine
-- una de las dos transacciones fallará por stock insuficiente
```

> **Ejemplo.** Dos usuarios intentan comprar a la vez la última unidad del producto P001. El aislamiento hace que el segundo espere a que el primero confirme, así que la última unidad no se vende dos veces (no hay **sobreventa**).

Durabilidad

Los cambios confirmados son permanentes: cuando se completa una transacción con commit, es imposible que

la base de datos la pierda.

- Paso 1: la transferencia de 1.000 € de A123 a B456 ejecuta los dos `UPDATE`.
- Paso 2: el `COMMIT` devuelve "OK" y la app muestra "Transferencia realizada".
- Paso 3: un segundo después se va la luz; puede que los nuevos saldos aún no estén escritos en los ficheros de datos.
- Paso 4: al reiniciar, el SGBD reaplica el log: A123 tiene 4.000 € y B456 3.000 €, exactamente lo que se le dijo al cliente.

```
-- Confirmación de pago
BEGIN TRANSACTION;
UPDATE pedidos SET estado = 'PAGADO' WHERE pedido_id = 12345;
INSERT INTO pagos (pedido_id, monto, fecha) VALUES (12345, 250.00, NOW());
COMMIT; -- ¡Confirmado!
```

> **Ejemplo.** El pago de 250,00 € del pedido 12345 sigue registrado tras un corte de luz que ocurre justo después del `COMMIT`. Antes del `COMMIT`, el mismo corte lo desharía (atomicidad).

> **Error frecuente.** Pensar que durabilidad significa "los datos ya están en los ficheros de datos". Lo que garantiza es que una transacción confirmada sobrevive a cualquier fallo; cómo lo consigue (log, réplicas) es cosa del SGBD.

Niveles de abstracción

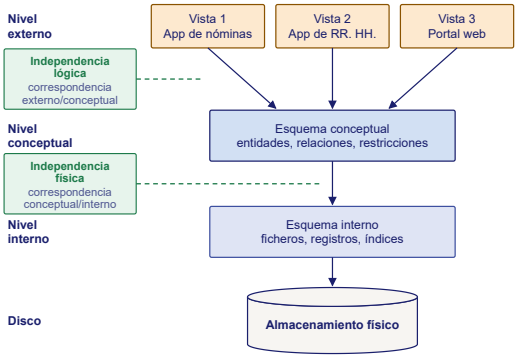
La arquitectura ANSI/X3/SPARC

El estándar, también conocido como arquitectura de tres esquemas, que separa una base de datos en nivel externo, nivel conceptual y nivel interno.

- **Nivel externo** (vista de los usuarios): vistas parciales de la base de datos que se muestran a los usuarios y/o aplicaciones.
- **Nivel conceptual** (vista lógica global): el esquema completo; refleja la estructura y relaciones existentes entre los datos del mundo real que se van a guardar en la base de datos, aislando entre sí los niveles externo e interno.
- **Nivel interno** (vista física): especifica qué, cómo y dónde se van a almacenar los datos físicamente en disco.

Los tres niveles en la base de datos de una universidad

Nivel	Qué contiene
Externo	Vista de la profesora: nombre y nota de los alumnos de su asignatura. Vista de secretaría: nombre, DNI y tasas pagadas, sin notas.
Conceptual	`ALUMNO(alumno_id, nombre, dni, email)`, `ASIGNATURA(codigo, titulo)`, `MATRICULA(alumno_id, codigo, nota)` con sus claves y restricciones.
Interno	`MATRICULA` almacenada como árbol B+ sobre `(alumno_id, codigo)` en bloques de 8 KB, en el fichero `matric.dat` de un SSD, con un índice adicional sobre `codigo`.



Varias vistas externas sobre un único esquema conceptual, almacenado mediante un único esquema interno. Las fronteras entre niveles dan la independencia lógica y la física.

> **Ejemplo.** Cuando una profesora abre "Mis alumnos" ve 3 columnas y 45 filas; no sabe que `MATRICULA` tiene 60.000 filas ni que vive en un árbol B+ sobre un SSD.

Independencia lógica y física

Cambiar un nivel no obliga a cambiar el nivel superior: la independencia física separa el nivel interno del conceptual; la independencia lógica separa el nivel conceptual del externo.

```
-- Independencia física: un índice nuevo cambia CÓMO se guardan y encuentran los datos
CREATE INDEX idx_matricula_codigo ON matricula (codigo);
-- La app de los profesores ejecuta exactamente el mismo SELECT, solo que más rápido.

-- Independencia lógica: el esquema conceptual crece
ALTER TABLE alumno ADD telefono VARCHAR(20);
-- La vista que usan los profesores no menciona telefono, así que sigue funcionando:
CREATE VIEW mis_alumnos AS
SELECT a.nombre, m.nota
```

```
FROM alumno a JOIN matricula m ON m.alumno_id = a.alumno_id
WHERE m.codigo = 'DB241';
```

> **Ejemplo.** Pasar `MATRICULA` de un HDD a un SSD, o añadir `idx_matricula_codigo`, no cambia ni una línea de la aplicación (física). Añadir la columna `telefono` a `alumno` no rompe la vista `mis_alumnos` (lógica).

> **Error frecuente.** Confundir las dos. Recuerda: **física** = interno ↔ conceptual (cambios de almacenamiento); **lógica** = conceptual ↔ externo (cambios de esquema).

Almacenamiento y eficiencia

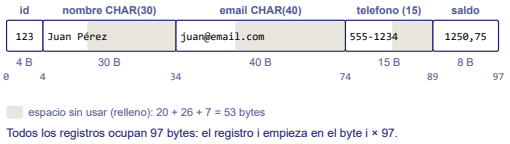
Ficheros, registros y registros de longitud fija

La base de datos se almacena como una colección de ficheros; cada fichero es una secuencia de registros, y los registros se componen de múltiples campos. Los registros pueden ser de longitud fija o variable.

- **Predecible**: todos los registros ocupan el mismo espacio.
- **Acceso rápido**: es fácil calcular la posición de cualquier registro.
- **Simplicidad**: gestión de memoria más sencilla.
- **Desperdicio**: espacios no utilizados en campos de contenido variable.
- **Límites rígidos**: no se puede exceder el tamaño máximo.

Disposición en bytes de un registro de `clientes_fijo` (97 bytes)

Campo	Tipo	Bytes	Desplazamiento
`id`	`INT`	4	0–3
`nombre`	`CHAR(30)`	30	4–33
`email`	`CHAR(40)`	40	34–73
`telefono`	`CHAR(15)`	15	74–88
`saldo`	`DECIMAL(10,2)`	8	89–96



Registros de longitud fija: cada campo tiene su anchura máxima, así que cada registro empieza en un múltiplo de 97 bytes.

```
CREATE TABLE clientes_fijo (
  id       INT,            -- 4 bytes
  nombre   CHAR(30),
  email    CHAR(40),
  telefono CHAR(15),
  saldo    DECIMAL(10,2)   -- 8 bytes
);
-- Total por registro: 97 bytes (siempre)
```

> **Ejemplo.** El registro número 1.000 (contando desde 0) empieza en el byte 1.000 × 97 = **97.000**, sin necesidad de leer los anteriores. Pero el nombre "Ana" usa 3 de sus 30 bytes: 27 bytes son relleno.

Registros de longitud variable

Campos como VARCHAR ocupan solo los bytes que necesitan más una marca de longitud, así que los registros tienen tamaños distintos.

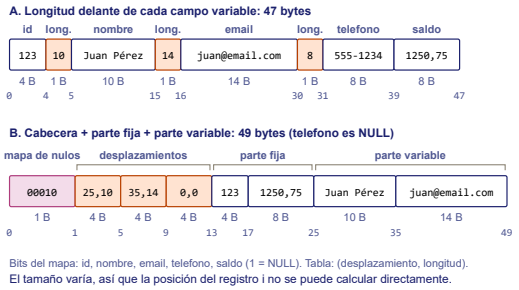
- **Eficiencia**: no desperdicia espacio.
- **Flexibilidad**: los campos pueden crecer según necesidad.
- **Realismo**: mejor para datos del mundo real.
- **Complejidad**: gestión más difícil.
- **Acceso lento**: no se puede calcular la posición directamente; hay que leer las longitudes (o mantener un directorio de posiciones).

Un registro: (7, 'Ana Ruiz', 'ana@cunef.edu', '600123456', 150.00), con 1 byte de longitud antes de cada VARCHAR

Campo	Se guarda como	Bytes
`id`	7	4

Campo	Se guarda como	Bytes
`nombre`	longitud 8 + "Ana Ruiz"	1 + 8 = 9
`email`	longitud 13 + "ana@cunef.edu"	1 + 13 = 14
`telefono`	longitud 9 + "600123456"	1 + 9 = 10
`saldo`	150.00	8
Total		**45** (frente a 97 en la tabla fija)

```
CREATE TABLE clientes_variable (
  id       INT,            -- 4 bytes
  nombre   VARCHAR(100),   -- 1-100 bytes + longitud
  email    VARCHAR(255),   -- 1-255 bytes + longitud
  telefono VARCHAR(20),    -- 1-20 bytes + longitud
  saldo    DECIMAL(10,2)   -- 8 bytes
);
```



Bits del mapa: id, nombre, email, telefono, saldo (1 = NULL). Tabla: (desplazamiento, longitud).
El tamaño varía, así que la posición del registro i no se puede calcular directamente.

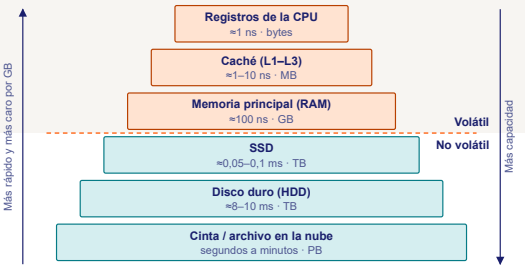
Registros de longitud variable: cada VARCHAR va precedido de su longitud, así que los registros tienen tamaños distintos y hay que recorrerlos para localizarlos.

> **Ejemplo.** El siguiente registro, de "Bartolomé Fernández-Villaverde", es más largo, así que el registro 2 no empieza en un desplazamiento fijo. Si Ana cambia su email por uno más largo, su registro crece y puede que haya que moverlo.

Soportes de almacenamiento y bloques

Las bases de datos deben residir en almacenamiento no volátil; como ese almacenamiento es lento, los datos se leen y escriben en bloques grandes.

> **Ejemplo.** Con registros de 97 bytes y bloques de 4 KB (4.096 bytes), en un bloque caben ⌊4.096 / 97⌋ = **42 registros**. Leer 4.200 clientes de forma secuencial cuesta 100 operaciones de E/S en lugar de 4.200.



Cuanto más cerca de la CPU, más rápida, pequeña, cara y volátil es la memoria; la base de datos vive en los niveles no volátiles y se guarda en caché en la RAM.

Tiempo de respuesta: Ts = α + βb

El tiempo de leer o escribir un bloque es un tiempo de búsqueda fijo más un tiempo de transferencia proporcional al tamaño del bloque.

Ejemplo resuelto: 10 millones de registros de transacciones, 100.000 consultas aleatorias por hora, registros de 2 KB (b = 0,002 MB)

	HDD (α = 10 ms, 80 MB/s)	SSD (α = 0,05 ms, 300 MB/s)
Tiempo por consulta	T = 10 ms + $(80\ \text{MB/s})^{-1}$ × 0,002 MB = 10 ms + 0,025 ms ≈ **10 ms**	T = 0,05 ms + $(300\ \text{MB/s})^{-1}$ × 0,002 MB = 0,05 ms + 0,0067 ms ≈ **0,05 ms**
Carga por hora	100.000 × 10 ms = 1.000.000 ms = **1.000 s** de disco por hora	100.000 × 0,05 ms = 5.000 ms = **5 s** por hora
¿Puede con la carga?	Viable: usa 1.000 / 3.600 = **27,78 %** del tiempo	Muy viable: usa 5 / 3.600 = **0,1389 %** del tiempo

> **Ejemplo.** En el ejemplo resuelto el término de transferencia es minúsculo: $(80\ \text{MB/s})^{-1}$ × 0,002 MB = 0,000025 s = 0,025 ms frente a 10 ms de búsqueda. Duplicar el registro a 4 KB solo añadiría otros 0,025 ms en el HDD: por eso compensa leer bloques más grandes.

> **Error frecuente.** Mezclar unidades. β debe ir en tiempo por MB (o por KB) y b en la misma unidad: 4 KB = 0,004 MB, y 1/(100 MB/s) = 0,01 s/MB = 10 ms/MB.

Organización de ficheros e índices

Las organizaciones de ficheros de un vistazo

La organización de un fichero es cómo se distribuye y localiza la información dentro de él: secuencial, heap, hash, árbol B+, agrupada (clustered) o ISAM.

Resumen de las seis organizaciones

Organización	Descripción	Ventajas	Inconvenientes	Casos de uso
Secuencial	Los registros se almacenan **uno tras otro** en orden; nuevos registros al final o con reordenación completa	Rápido para datos contiguos; simple de implementar; almacenamiento barato	Tiempo de lectura alto; el borrado causa fragmentación; ordenación muy lenta	Archivos de respaldo, cargas masivas, datos históricos
Heap	Registros **desordenados** con ID único; cada registro tiene una dirección (bucket address); inserción al final	Inserción muy rápida; bueno para BD pequeñas; no requiere ordenación	Lento para BD grandes; desperdicia memoria; búsqueda secuencial	BD pequeñas, logs de sistema, inserción masiva
Hash	Usa una **función hash** para calcular la ubicación del registro a partir de su clave primaria	Acceso directo muy rápido; no necesita ordenación; tiempo constante	Puede borrar datos por error; memoria no consecutiva; colisiones hash	BD medianas, acceso por clave, transacciones frecuentes
Árbol B+	**Árbol equilibrado** donde los datos están en las hojas y los nodos internos son punteros para navegar	Búsqueda eficiente; árbol equilibrado; dinámico (crece/decrece); recorrido rápido	Coste de equilibrado; complejidad alta; overhead de punteros	BD grandes, muchas actualizaciones, consultas ordenadas
Agrupada (clustered)	**Combina múltiples tablas** en el mismo bloque usando un identificador común (índice o hash)	Excelente para JOINs; eficiente en relaciones 1:M; reduce accesos a disco	Malo para relaciones 1:1; no apto para BD grandes; ineficiente sin JOINs	Relaciones 1:M frecuentes, muchas operaciones JOIN, tablas relacionadas
ISAM	Crea un **archivo de índices** separado del archivo de datos; el índice apunta a la ubicación real del registro	Búsquedas por rangos; patrones de búsqueda; BD grandes	Almacenamiento extra para índices; los índices crecen con los datos; mantenimiento complejo	BD muy grandes, búsquedas complejas, consultas por rangos

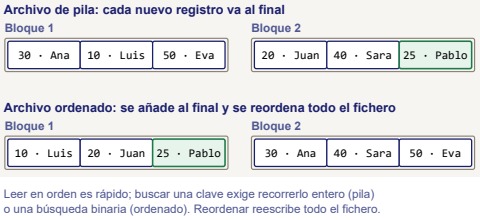
> **Ejemplo.** Una tienda con 2.000 productos que se compran por ID: hash. La misma tienda con 50 millones de líneas de pedido consultadas por rango de fechas: árbol B+. Una copia nocturna de toda la base de datos que solo se lee de principio a fin: secuencial.

Organización secuencial (Sequential File Organization)

Los registros se guardan uno tras otro. En un archivo de pila cada nuevo registro va siempre al final; en un archivo ordenado se añade al final y después se ordena el fichero.

- **Inserción**: archivo de pila, se escribe al final (barato); archivo ordenado, se añade y se reordena todo el fichero usando un fichero auxiliar (operación excesivamente lenta).
- **Búsqueda**: leer los registros en orden hasta encontrar la clave; de media, la mitad del fichero.
- **Ventajas**: rapidez siempre que se acceda a registros contiguos; método sencillo de implementar; los datos se pueden guardar en dispositivos de almacenamiento baratos.
- **Inconvenientes**: tiempo medio de lectura alto; el borrado puede provocar fragmentación interna del fichero; la ordenación requiere un tratamiento integral de todos los registros.
- **Aplicación**: ficheros de carga masiva de información en los que la rapidez de acceso y de las operaciones de actualización no sean factores críticos. Poco habituales en SGBD.

> **Ejemplo.** Un archivo ordenado de 1.000.000 de movimientos bancarios ordenados por fecha: imprimir marzo es rápido (registros contiguos), pero encontrar el movimiento 834.201 por su ID supone leer de media unos 500.000 registros, e insertar un movimiento del 2 de enero obliga a reescribir el fichero entero.



Leer en orden es rápido; buscar una clave exige recorrerlo entero (pila) o una búsqueda binaria (ordenado). Reordenar reescribe todo el fichero.

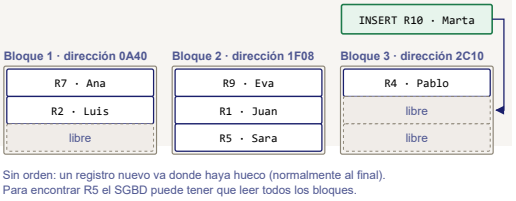
Archivo de pila: cada nuevo registro va al final. Archivo ordenado: se añade al final y después se ordena el fichero.

Organización heap (Heap File Organization)

Organización de archivo desordenado: cada registro tiene un ID único enlazado a una bucket address, y los nuevos registros se insertan al final del fichero sin reordenación.

- **Punteros**: cada registro se enlaza con su ubicación en memoria.
- **Sin ordenación**: los registros no siguen ningún orden particular.
- **Flexibilidad**: cualquier bloque disponible puede usarse para nuevos registros; el SGBD gestiona automáticamente la asignación, el almacenamiento y la administración.
- **Inserción**: se añade al final (o en cualquier bloque libre), sin reorganizar: muy rápida.
- **Búsqueda**: recorrer el fichero; rápida en una base de datos pequeña, lenta en una grande.
- **Inconveniente**: el efecto "memory wastage", al desperdiciar los huecos que dejan los bloques que almacenan registros que no se ajustan al tamaño máximo del bloque.
- **Aplicación**: bases de datos pequeñas con constantes operaciones de actualización o inserción; cargas masivas, si se dispone de suficiente memoria.

> **Ejemplo.** Un servidor web escribe 5.000 líneas de log por minuto en una tabla heap `log_accesos`: cada línea simplemente se añade. Pero buscar "todas las peticiones de la IP 10.0.0.7" en una tabla de 200 millones de líneas obliga a leer todos los bloques.



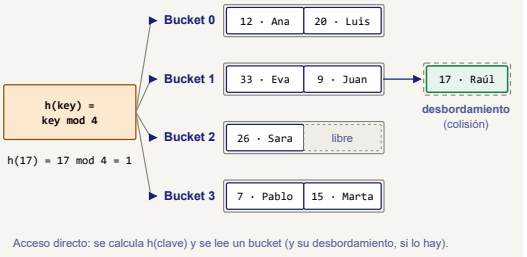
Fichero heap: los registros van a cualquier bloque libre en orden de llegada; cada uno tiene su bucket address, pero no se mantiene ningún orden.

Organización hash (Hash File Organization)

Una función hash genera la bucket address de cada registro a partir de su clave primaria; las localizaciones que genera se denominan data buckets o data blocks.

- **Inserción**: se calcula h(clave) y se escribe el registro en ese bucket. La función puede ser una función matemática simple o una compleja; calcularla lleva un tiempo constante que no depende del estado de la memoria o del archivo.
- **Búsqueda por clave**: se vuelve a calcular h(clave) y se va directamente al bucket: sin ordenación y sin recorridos.
- **Inconvenientes**: eliminación accidental de datos si la clave está mal elegida (valores repetidos dan el mismo bucket); la memoria no se usa de manera eficiente porque los registros no están en ubicaciones consecutivas; **colisiones**; y no sirve bien para búsquedas por rango, porque claves vecinas acaban en buckets sin relación.
- **Aplicación**: bases de datos medianas con constantes operaciones de actualización o inserción y acceso por clave.

> **Ejemplo.** Con 5 buckets y h(id) = id mod 5: el empleado 1027 va al bucket 1027 mod 5 = **2** y el empleado 1030 al bucket **0**. Encontrar el 1027 lee solo el bucket 2. Pero "empleados con id entre 1000 y 1100" tiene que leer los 5 buckets. Usar `employee_name` como clave hash sería arriesgado: dos empleados llamados "Luis Pérez" caen en el mismo bucket y un borrado por nombre podría eliminar a los dos; combinar el nombre con el departamento o el NIF lo evita.



La función hash convierte cada clave en una bucket address; el registro se guarda y después se encuentra en ese bucket.

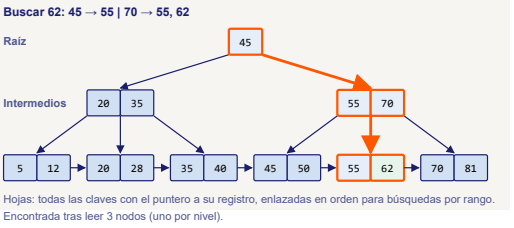
Organización en árbol B+ (B+ tree File Organization)

Un árbol de búsqueda equilibrado en el que cada nodo puede tener muchos descendientes: todos los registros están en los nodos hoja, y los nodos intermedios actúan como punteros que conducen a ellos.

- **Búsqueda**: desde la raíz, sigue en cada nivel el puntero cuyo rango contiene la clave, hasta llegar a una hoja. Coste = altura del árbol.
- **Inserción/borrado**: se baja hasta la hoja correcta; si se desborda, se divide y se sube una clave. El árbol se mantiene **equilibrado**: todas las ramas tienen la misma profundidad.
- **Ventajas**: búsqueda eficiente; las eliminaciones, inserciones o actualizaciones no afectan al rendimiento general; recorrido fácil y relativamente rápido; el tamaño crece o se reduce dinámicamente.

- **Inconveniente**: el coste de las operaciones de equilibrado para que la profundidad de todas las ramas se mantenga constante.
- **Aplicación**: grandes bases de datos con altas tasas de actualización.

> **Ejemplo.** Una tabla de 1.000.000 de clientes. Un recorrido completo compara hasta 1.000.000 de filas; la búsqueda binaria en un fichero ordenado, unos $\log_2 1.000.000 \approx$ **20** pasos. Un árbol B+ con 100 claves por nodo solo necesita **3 niveles** (100 × 100 × 100 = 1.000.000), es decir, 3 lecturas de bloque, y "clientes con id entre 5.000 y 5.099" continúa por las hojas encadenadas.



Los nodos intermedios solo guían la búsqueda; todos los registros se alcanzan a la misma profundidad, en las hojas enlazadas.

Organización agrupada (Clustered File Organization)

Dos o más registros/tablas se combinan en un solo bloque teniendo en cuenta un identificador/índice o el resultado de una función hash: el clúster ID.

- **Indexed Clusters**: el clúster ID se obtiene directamente a partir del valor identificador/índice del fichero.
- **Hash Clusters**: el clúster ID se obtiene aplicando una función hash sobre los índices.
- **Inserción**: la nueva fila va al bloque de su clúster ID, junto a sus filas relacionadas de la otra tabla.
- **Ventajas**: buen rendimiento ante operaciones de join entre tablas; muy eficiente para implementar relaciones 1:M; reduce accesos a disco.
- **Inconvenientes**: ineficiente cuando no son necesarias operaciones frecuentes de join y en relaciones uno a uno; no es adecuado para grandes bases de datos.
- **Aplicación**: relaciones 1:N, por ejemplo muchos estudiantes pueden optar a un curso, y uso frecuente de join.

> **Ejemplo.** El curso DB241 y sus 45 estudiantes se guardan en el mismo bloque, con clúster ID `curso_id = DB241`. "Lista los estudiantes de DB241" lee un bloque en lugar de un bloque para el curso más hasta 45 bloques dispersos para los estudiantes.



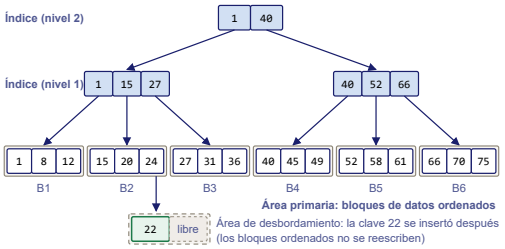
Las filas de dos tablas que comparten clúster ID viven en el mismo bloque, así que el join ya está hecho en disco.

ISAM (Indexed Sequential Access Method)

Una organización de ficheros avanzada: a partir del ID de cada registro se genera una entrada en un archivo de índices separado, que apunta a la bucket address del bloque de memoria que contiene el registro.

- **Búsqueda**: se busca la clave (o el inicio del rango) en el archivo de índices, pequeño y ordenado, se sigue el puntero al bloque de datos y se continúa leyendo de forma secuencial.
- **Inserción**: se escribe el registro y se añade su entrada al archivo de índices.
- **Ventajas**: facilita las búsquedas a partir de rangos de valores o patrones de búsqueda.
- **Inconvenientes**: necesita almacenamiento extra para los índices; cuando los registros aumentan en número, también aumentan los índices; mantenimiento más complejo.
- **Aplicación**: bases de datos de gran tamaño, dada la eficiencia de los índices para el acceso rápido a los datos.

> **Ejemplo.** "Todos los estudiantes cuyo apellido empiece por Gar": el índice encuentra la primera entrada ≥ "Gar" (García, Ana → bloque 812), y la lectura sigue por García, Garrido... hasta el primer apellido que ya no empieza por "Gar". Solo se leen unos pocos bloques de entre miles.



El archivo de índices guarda las claves ordenadas con punteros a los bloques de datos; una búsqueda por rango encuentra su inicio en el índice y después lee secuencialmente.

Cómo elegir la organización de ficheros

Elige la organización según el tamaño de la base de datos y su operación principal.

Según la operación principal (según el tamaño: pequeña < 10K registros → heap, alternativa secuencial; mediana 10K–1M → hash, alternativa árbol B+; grande > 1M → árbol B+, alternativa ISAM)

Operación principal	Mejor opción	¿Por qué?
Inserción masiva	Heap	No reorganiza, muy rápido
Búsqueda por ID	Hash	Acceso directo constante
Búsquedas ordenadas	Árbol B+	Datos naturalmente ordenados
JOINs frecuentes	Agrupada (clustered)	Tablas juntas físicamente
Búsquedas por rango	ISAM	Índices optimizados
Solo lectura	Secuencial	Simple y eficiente

> **Ejemplo.** Una cadena de gimnasios: el registro de los tornos (2 millones de inserciones al día, casi nunca leído) → heap; buscar a un socio por su número de tarjeta entre 300.000 → hash; la tabla de 40 millones de visitas consultada "por mes" y actualizada constantemente → árbol B+; cada clase con sus reservas, que siempre se leen juntas → agrupada.

> **Error frecuente.** Elegir hash para una tabla que se consulta sobre todo por rangos ("pedidos del 1 al 15 de mayo"). El hash dispersa las claves vecinas, así que cada consulta por rango se convierte en un recorrido completo.

Ciclo de vida de los datos

Modelos de datos y para qué sirven

Un modelo de datos es un conjunto de pasos, tareas y técnicas de representación para obtener estructuras de datos que resuelvan un problema de forma metódica y sencilla.

- **Control de errores**: se detectan y corrigen antes de la implementación.
- **Independencia**: las estructuras de datos se mantienen separadas del entorno físico y lógico.
- **Mejora del mantenimiento**: facilita la gestión y actualización del sistema.
- **Mejor comprensión del problema**: los modelos ayudan a entender claramente las necesidades.
- **Facilita la comunicación**: mejora la colaboración entre los miembros del equipo de desarrollo.

> **Ejemplo.** Al dibujar el modelo de una cadena de gimnasios, el equipo se da cuenta de que un socio puede pertenecer a varios gimnasios. Corregir una línea del diagrama cuesta minutos; descubrirlo después del lanzamiento, con 300.000 socios guardados con un único `gimnasio_id`, obliga a migrar datos y reescribir consultas.

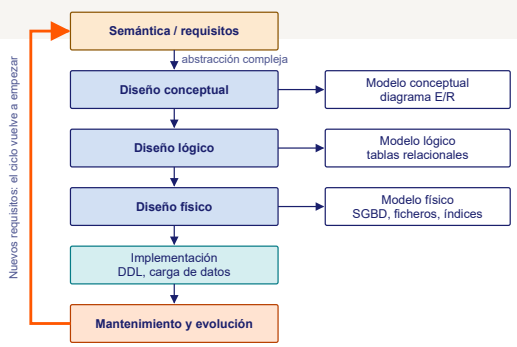
El ciclo de vida del modelado de datos

Desde la semántica/requisitos, pasando por el diseño conceptual, lógico y físico, cada uno con su modelo; las técnicas de ingeniería del software cierran el ciclo desde el diseño físico hasta los requisitos.

Ejemplo conductor: una cadena de gimnasios, "FitRed"

Etapa	Qué se hace	FitRed

Etapa	Qué se hace	FitRed
Semántica / requisitos	Entrevistar a los usuarios, recoger las reglas de negocio	Los socios se apuntan a uno o más gimnasios; los gimnasios ofrecen clases; un socio reserva clases; una clase tiene como máximo 20 plazas; la dirección necesita las visitas por mes
Diseño conceptual → modelo conceptual	Diagrama E/R, independiente del SGBD	Entidades SOCIO, GIMNASIO, CLASE; relación RESERVA (SOCIO M:N CLASE); OFRECE (GIMNASIO 1:N CLASE)
Diseño lógico → modelo lógico	Tablas, claves, claves foráneas	`socio(socio_id PK, nombre, email)`, `clase(clase_id PK, gimnasio_id FK, inicio, plazas)`, `reserva(socio_id FK, clase_id FK, PK(socio_id, clase_id))`
Diseño físico → modelo físico	SGBD, tipos, índices, organización de ficheros	PostgreSQL sobre SSD; `CHECK (plazas <= 20)`; índice árbol B+ sobre `reserva(clase_id)`; tabla heap para los registros de los tornos
Mantenimiento (vuelta a los requisitos)	Monitorizar, optimizar, incorporar nuevas necesidades	FitRed añade entrenadores personales: nueva entidad ENTRENADOR y relación ENTRENA, y el ciclo vuelve a empezar



Requisitos → diseño conceptual → lógico → físico, cada uno con su modelo; el mantenimiento vuelve a los requisitos.

> **Ejemplo.** En FitRed, la regla "una clase tiene como máximo 20 plazas" aparece en los requisitos, se convierte en un atributo `plazas` de CLASE en el modelo conceptual, en una columna en el modelo lógico y en una restricción `CHECK` más una transacción en el diseño físico.